université
PARIS-SACLAY

QUERYING DATA WITH MONGODB

Combining Docker & Studio 3T

Author:
Akshay KHOKAN

Supervised by :
Nazim AGOULMINE

M2 GENIOMHE-AI

Table of contents

PREPROCESSING **2**
1. Creation of the base moviesref *2*
2. Information checking in each collection and differences *3*
EXERCISE 7 ..... **5**
Queries from the chosen collection *5*
a) All titles of movies 5
b) All the titles of movies released after 2000 and before 2019 6
c) The summary of Spider-Man movie 6
d) Who is the director of Gladiator 7
e) Titles of films with Kirsten Dunst 7
f) How many movies have a summary? 7
g) What are movies with actors' names that start with "Cl" ? 8
h) Movies that are neither dramas nor comedies 9
i) Titles of the films and the names of the actors 10
j) In which films is Clint Eastwood an actor but not a director 10
EXERCISE 8 ..... **11**
a) All titles of movies 11
b) All the titles of movies released after 2000 and before 2019 12
c) The summary of Spider-Man movie 13
d) Who is the director of Gladiator 13
e) Titles of films with Kirsten Dunst 14
f) How many movies have a summary? 14
g) What are movies with actors' names that start with "Cl" ? 15
h) Movies that are neither dramas nor comedies 16
i) Titles of the films and the names of the actors 17
j) In which films is Clint Eastwood an actor but not a director 18

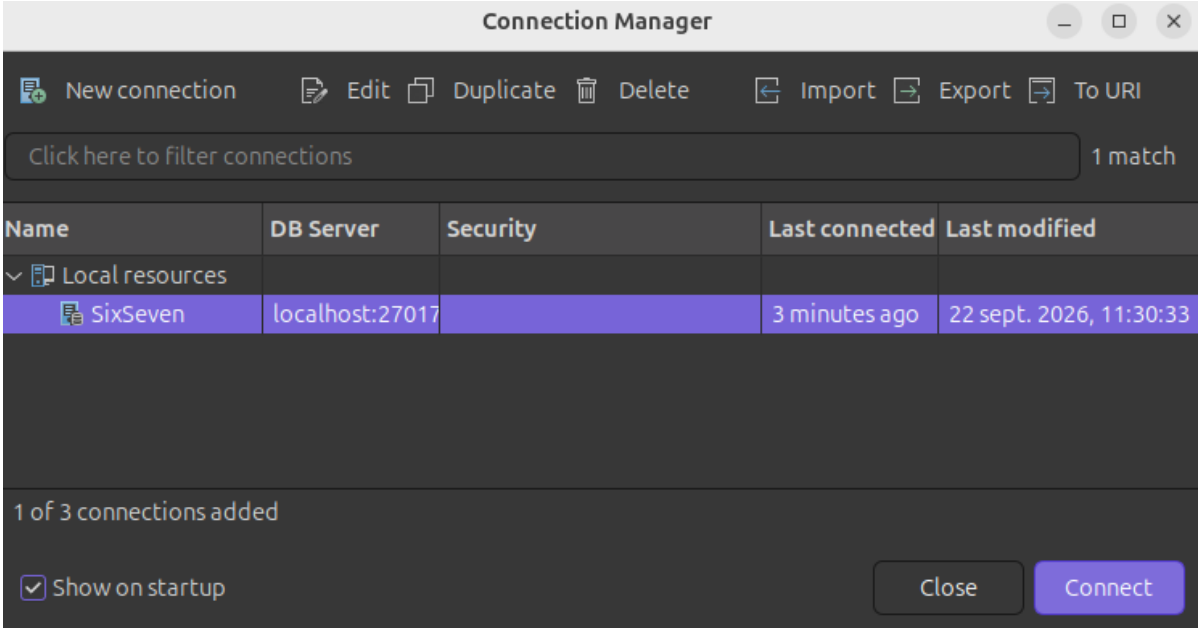
PREPROCESSING

1. Creation of the base moviesref

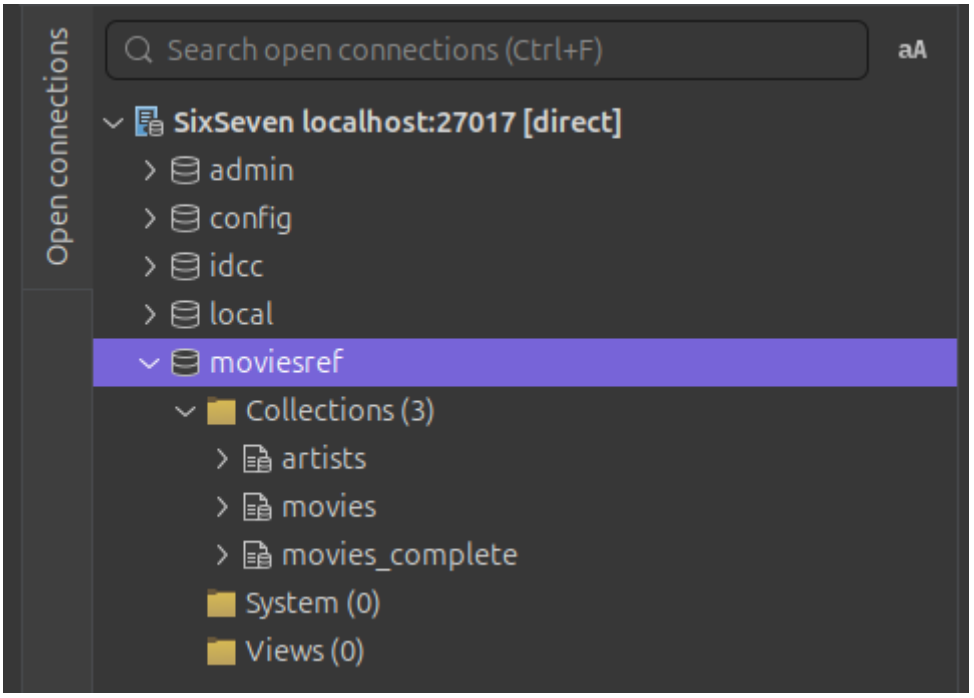
```shell
docker cp movies-JSON-refs.json mongodb:/tmp/
docker cp artistes_mongo.json mongodb:/tmp/
docker cp films.json mongodb:/tmp/

docker exec mongodb mongoimport -d moviesref -c movies_complete   --file
/tmp/films.json --jsonArray
docker exec mongodb mongoimport -d moviesref -c artists  --file
/tmp/artistes_mongo.json --jsonArray
docker exec mongodb mongoimport -d moviesref -c movies --file
/tmp/movies-JSON-refs.json --jsonArray
```

Movies, **Artists**, **Movies_complete** are 3 different collections in the **moviesref** database.



After using Docker, the collections can be visualized in Studio 3T by connecting to the moviesref database locally.



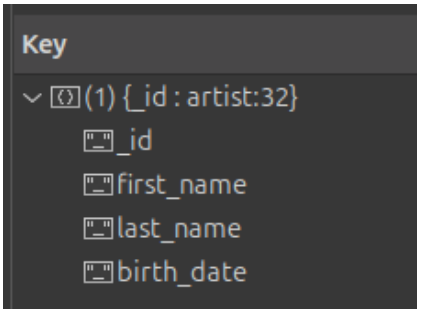
The connection confirms the creation of the base with Studio 3T (Using Docker is also an option but Studio 3T allows us to visualize the data more efficiently).

2. Information checking in each collection and differences

To check the information in each collection and the differences, the command lines here are used directly in Studio3T linked to Mongo.

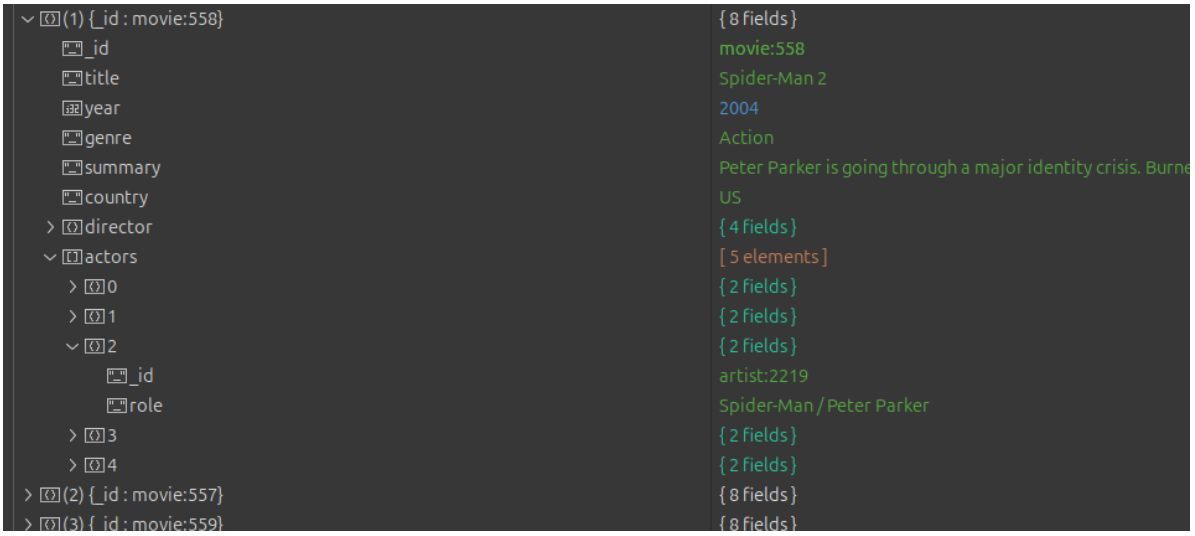
```
db.movies_complete.find({title:/Spider-Man/})
db.movies.find({title:/Spider-Man/})
db.artists.find({_id:"artist:2219"})
```

The goal here was to see the arborescence of each collection, the **Artists** one has the structure below.

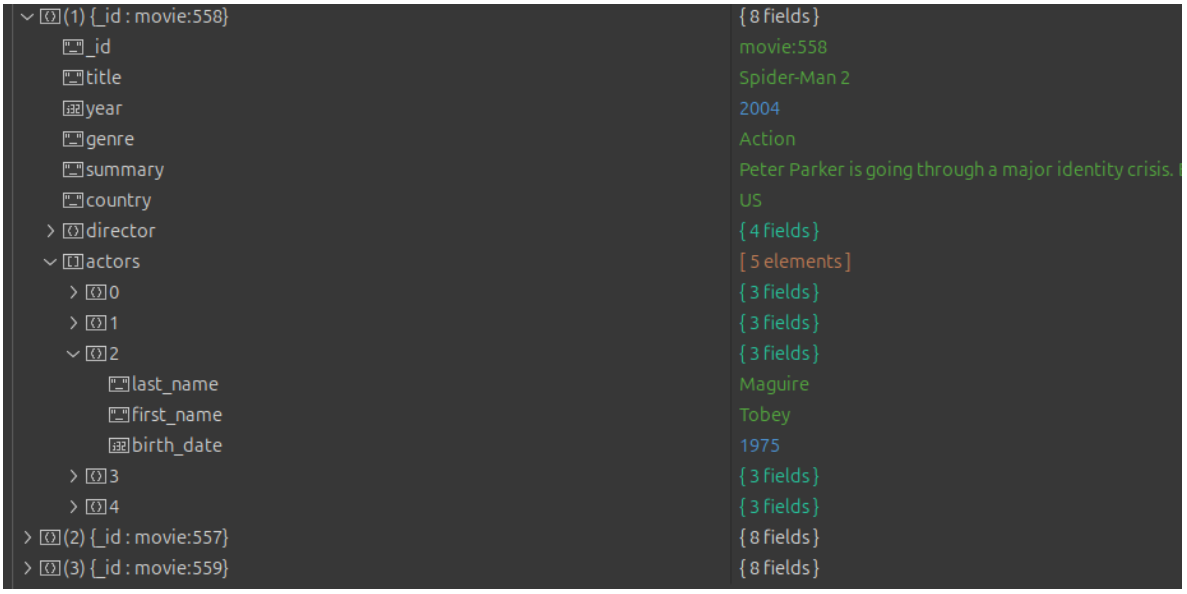


The **Movies** and **Movies_complete** are the same collections but slightly different. Indeed, the arborescence is the same, but the datasets are not. Using “Spider-Man”

as a key-word in the title confirms the difference (Spider-Man 2 is used here to compare both collections). Indeed, **Movies_complete** doesn't allow the lookup in the **Artists** collection for the actors, as there is no id for them. **Movies** contains the actors and their respective id, and it's an internal numbering related to **Artists**.



Movies collection arborescence



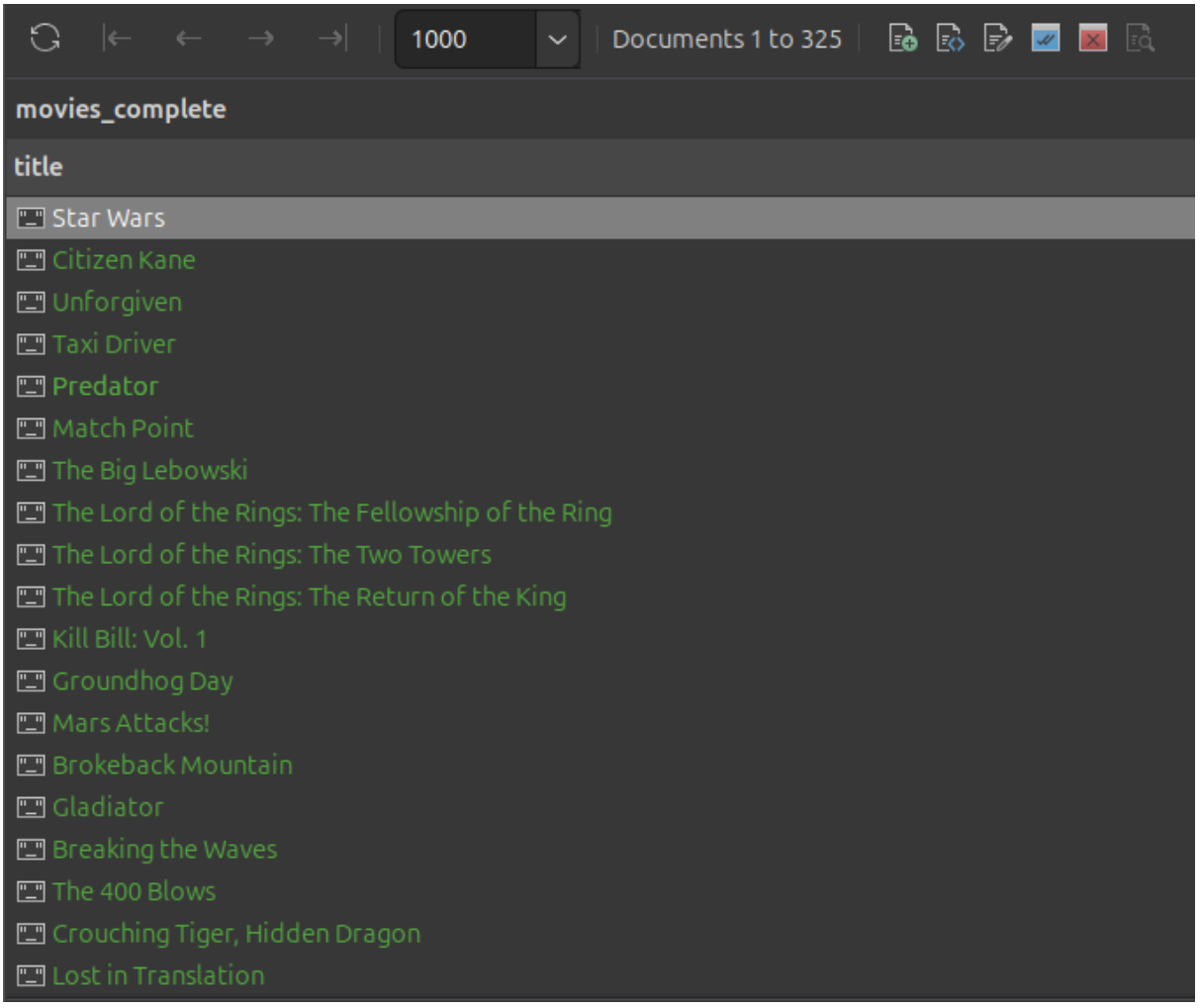
Movies_complete collection arborescence

EXERCISE 7

As seen above in the preprocessing step, both collections of movies are different. Moreover, no question in this exercise involves any look-up in another collection, thus **Movies_complete** will be used to answer the questions.

Queries from the chosen collection

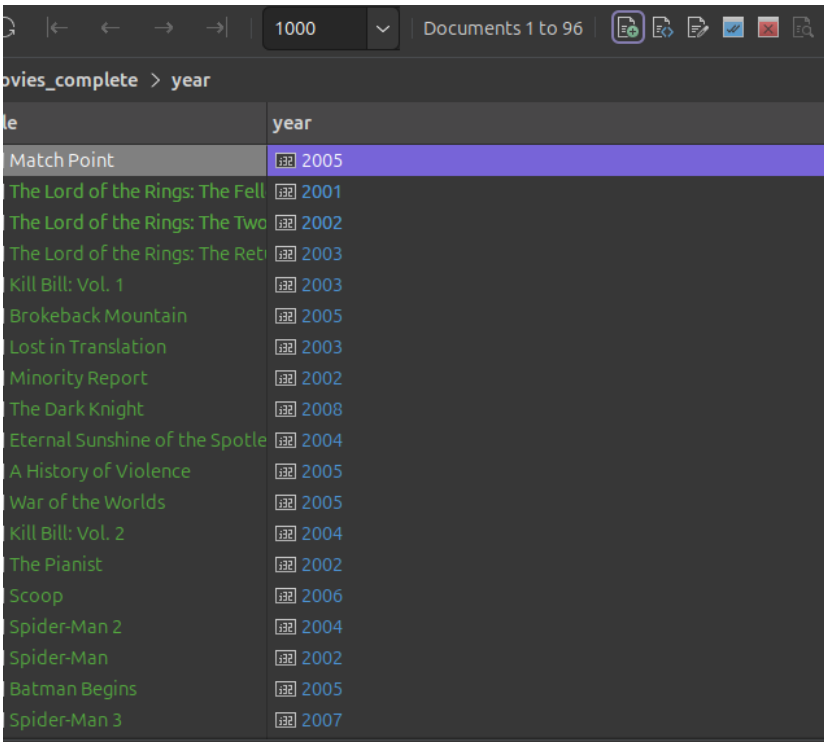
a) All titles of movies




```
db.movies_complete.find({}, {_id:0, title:1})
```

Putting 0 for _id in the projection allows the visualization of titles only.

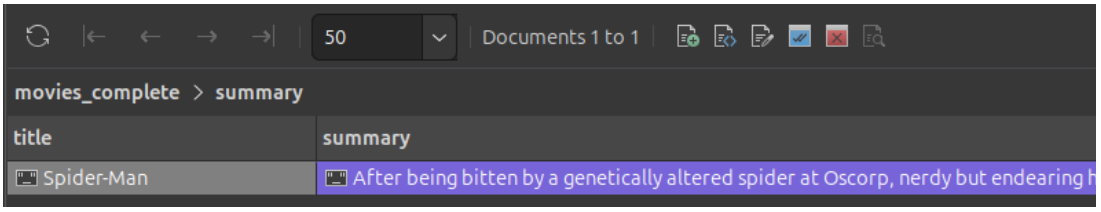
b) All the titles of movies released after 2000 and before 2019




```
db.movies_complete.find({year: {$gt:2000, $lt:2019}}, {_id: 0, title:1,
year:1})
```

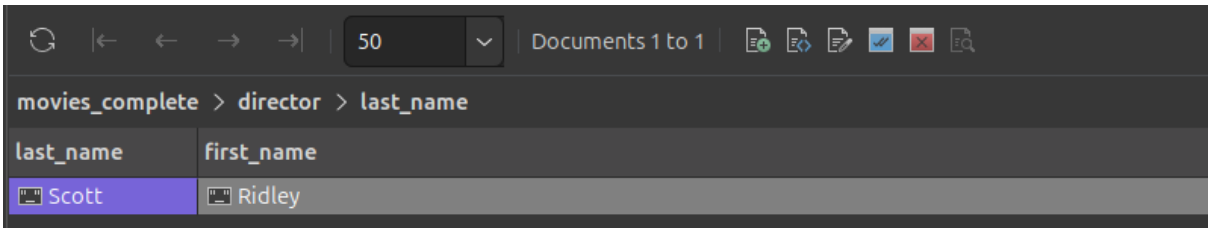
$gt, $lt being “greater than” and “lower than”, they were used to answer this question. 96 movies meet this condition.

c) The summary of Spider-Man movie




```
db.movies_complete.find({title: "Spider-Man"}, {_id: 0, title:1,
summary:1})
```

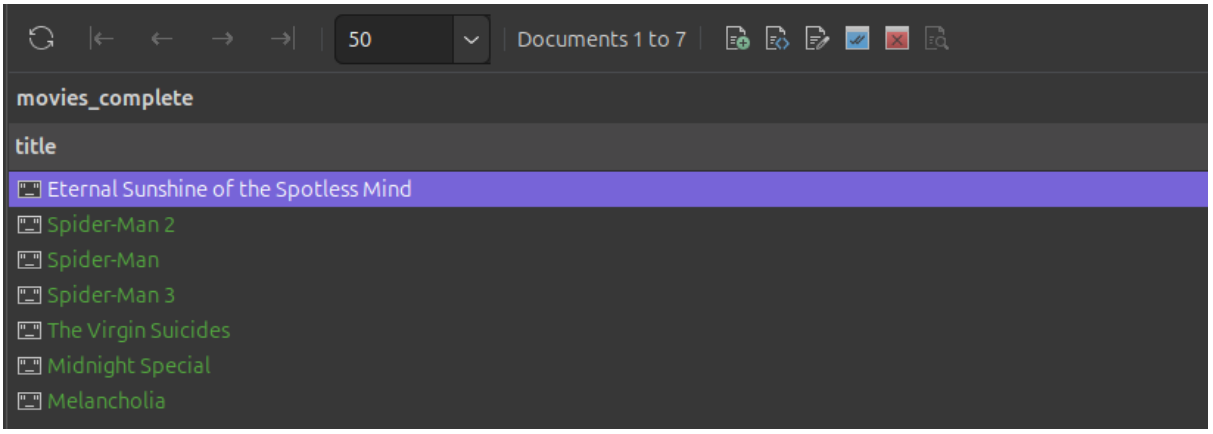
d) Who is the director of Gladiator




```
db.movies_complete.find({title: "Gladiator"}, {_id: 0,
“director.first_name”:1, “director.last_name”:1})
```

Putting the ***director*** key to 1 in the projection can be considered to get every information about the director. However, only the first name and the last name were taken here to answer the question.

e) Titles of films with Kirsten Dunst



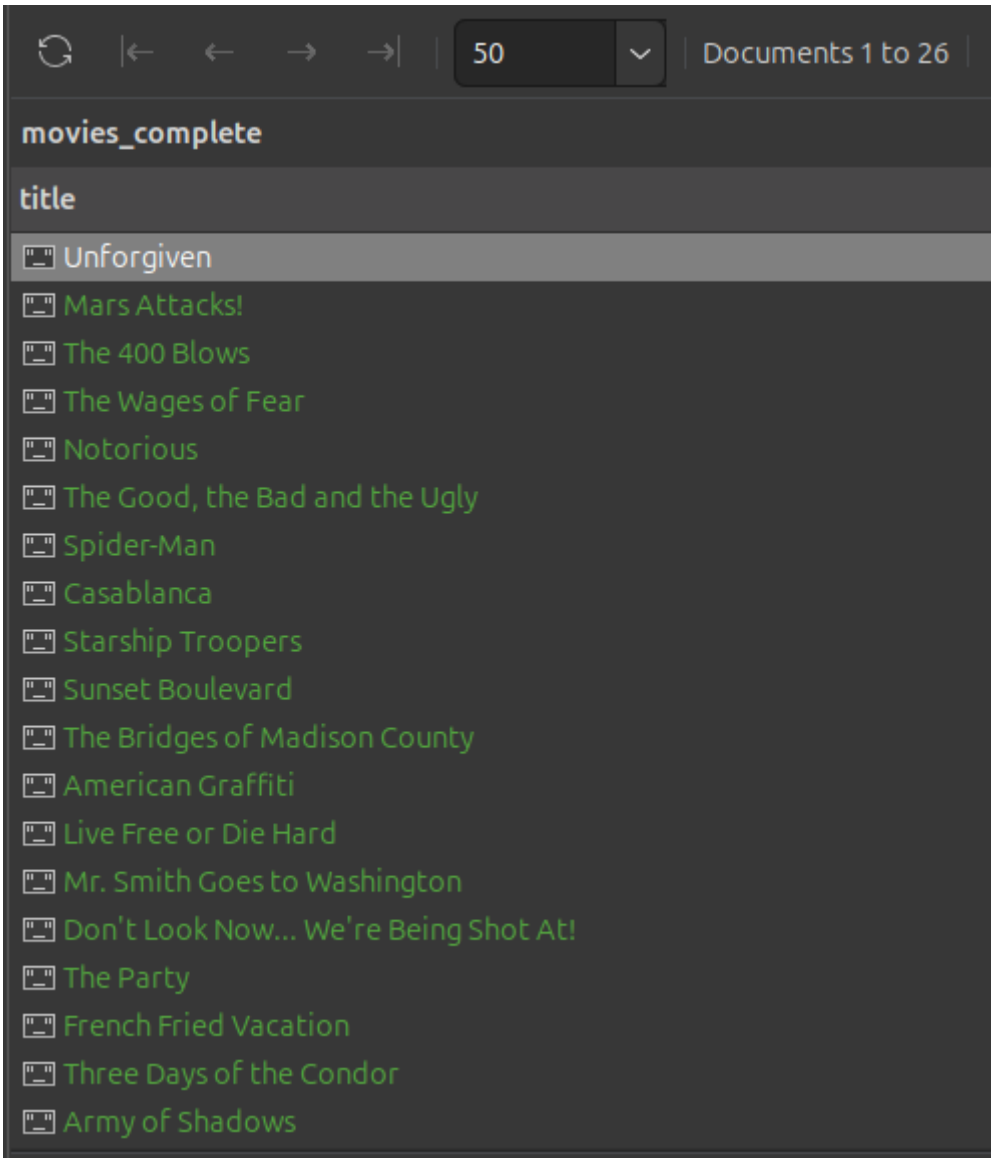

```
db.movies_complete.find({$and: [{"actors.last_name": "Dunst"},
{"actors.first_name": "Kirsten"}]}, {_id: 0, title: 1})
```

f) How many movies have a summary?

```
db.movies_complete.find({summary:{$ne: null}}).count()
```

325 out of 325 movies have a summary. No movies without a summary were found.

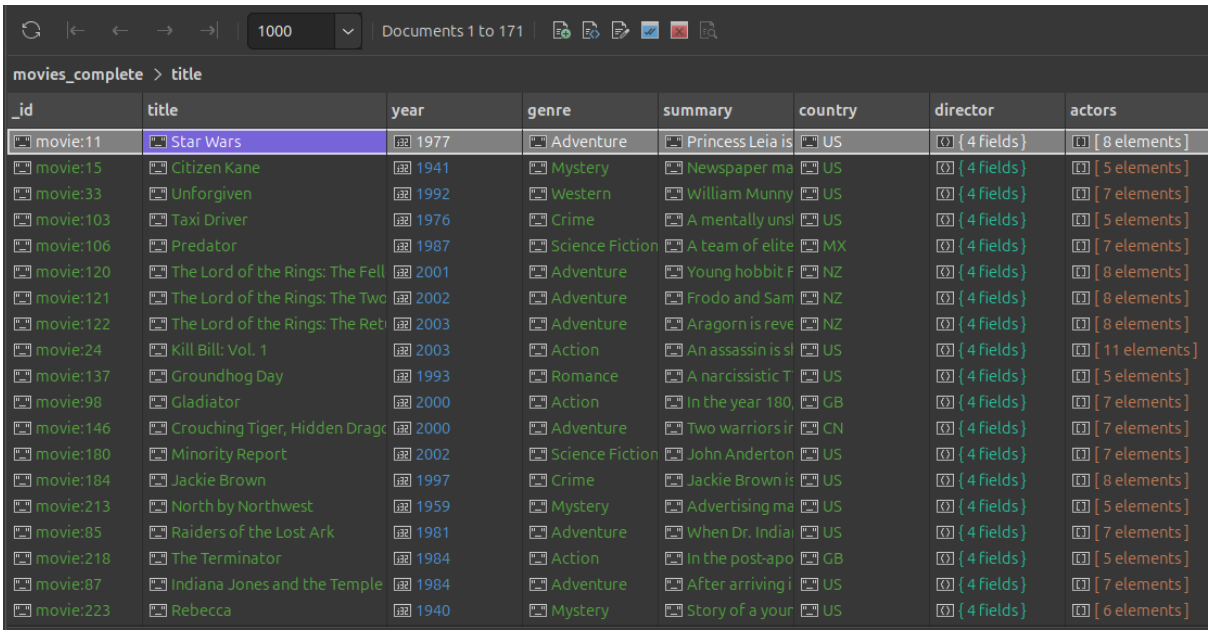
g) What are movies with actors' names that start with "Cl" ?




```
db.movies_complete.find({$or: [{"actors.last_name": /^Cl/},
{"actors.first_name": /^Cl/}]}, {_id: 0, title: 1})
```

With the OR condition, one movie is found when one of the actors has his last name or first name starting with "Cl", it is not necessary to retrieve every actor with this last/first name in every film. 26 movies meet this condition.

h) Movies that are neither dramas nor comedies.



1000 Documents 1 to 171

movies_complete > title

_id	title	year	genre	summary	country	director	actors
movie:11	Star Wars	1977	Adventure	Princess Leia is	US	{ 4 fields }	[8 elements]
movie:15	Citizen Kane	1941	Mystery	Newspaper ma	US	{ 4 fields }	[5 elements]
movie:33	Unforgiven	1992	Western	William Munny	US	{ 4 fields }	[7 elements]
movie:103	Taxi Driver	1976	Crime	A mentally unst	US	{ 4 fields }	[5 elements]
movie:106	Predator	1987	Science Fiction	A team of elite	MX	{ 4 fields }	[7 elements]
movie:120	The Lord of the Rings: The Fell	2001	Adventure	Young hobbit F	NZ	{ 4 fields }	[8 elements]
movie:121	The Lord of the Rings: The Two	2002	Adventure	Frodo and Sam	NZ	{ 4 fields }	[8 elements]
movie:122	The Lord of the Rings: The Retu	2003	Adventure	Aragorn is reve	NZ	{ 4 fields }	[8 elements]
movie:24	Kill Bill: Vol. 1	2003	Action	An assassin is sl	US	{ 4 fields }	[11 elements]
movie:137	Groundhog Day	1993	Romance	A narcissistic T	US	{ 4 fields }	[5 elements]
movie:98	Gladiator	2000	Action	In the year 180,	GB	{ 4 fields }	[7 elements]
movie:146	Crouching Tiger, Hidden Drago	2000	Adventure	Two warriors ir	CN	{ 4 fields }	[7 elements]
movie:180	Minority Report	2002	Science Fiction	John Anderton	US	{ 4 fields }	[7 elements]
movie:184	Jackie Brown	1997	Crime	Jackie Brown is	US	{ 4 fields }	[8 elements]
movie:213	North by Northwest	1959	Mystery	Advertising ma	US	{ 4 fields }	[5 elements]
movie:85	Raiders of the Lost Ark	1981	Adventure	When Dr. India	US	{ 4 fields }	[7 elements]
movie:218	The Terminator	1984	Action	In the post-apo	GB	{ 4 fields }	[5 elements]
movie:87	Indiana Jones and the Temple	1984	Adventure	After arriving i	US	{ 4 fields }	[7 elements]
movie:223	Rebecca	1940	Mystery	Story of a your	US	{ 4 fields }	[6 elements]

```
db.movies_complete.find({$nor: [{genre: "Comedy"}, {genre: "Drama"}]})
```

171 movies are neither dramas nor comedies.

i) Titles of the films and the names of the actors.




```
db.movies_complete.find({}, {_id: 0, title:1, "actors.first_name": 1,
"actors.last_name":1})
```

The table format was not showing the data as intended, so the visualization got replaced by the .json format :

```
[
 {title : Starwars, actors : [{"last_name":...}]},
 {title : Spider-Man, actors : [{"last_name": Maguire, ...}]}
]
```

j) In which films is Clint Eastwood an actor but not a director

The Good, the Bad and the Ugly - Sergio Leone

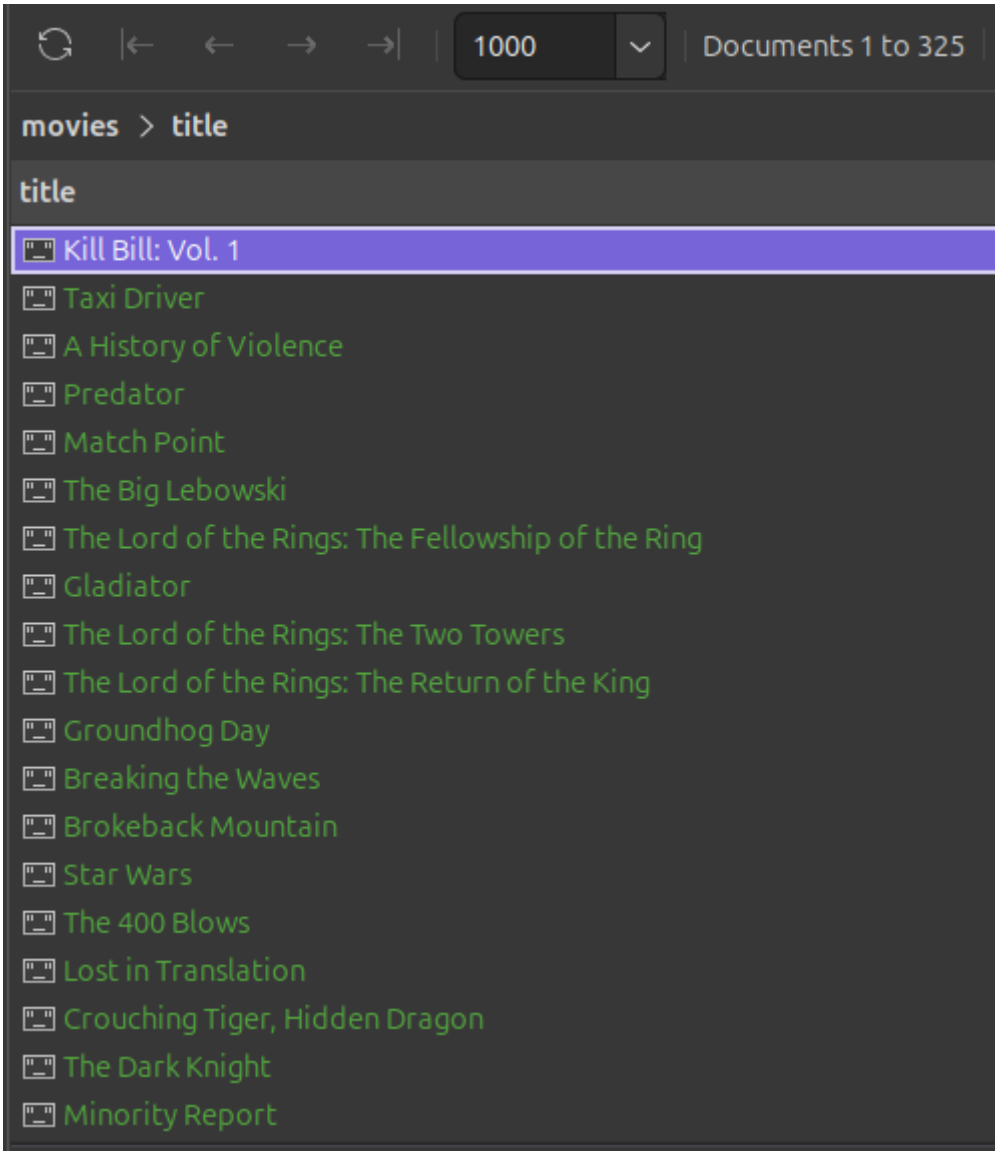
```
db.movies_complete.find({$and: [{"actors.last_name": "Eastwood"},
{"actors.first_name": "Clint"}],
                        $or: [{"director.last_name": {$ne:"Eastwood"}},
{"director.first_name": {$ne:"Clint"}}]},
                        {_id: 0, title:1})
```

An OR condition was used here for the director's name, because if the last name is not Eastwood, then the movies we are searching for can be retrieved. An AND condition could have been used as well.

EXERCISE 8

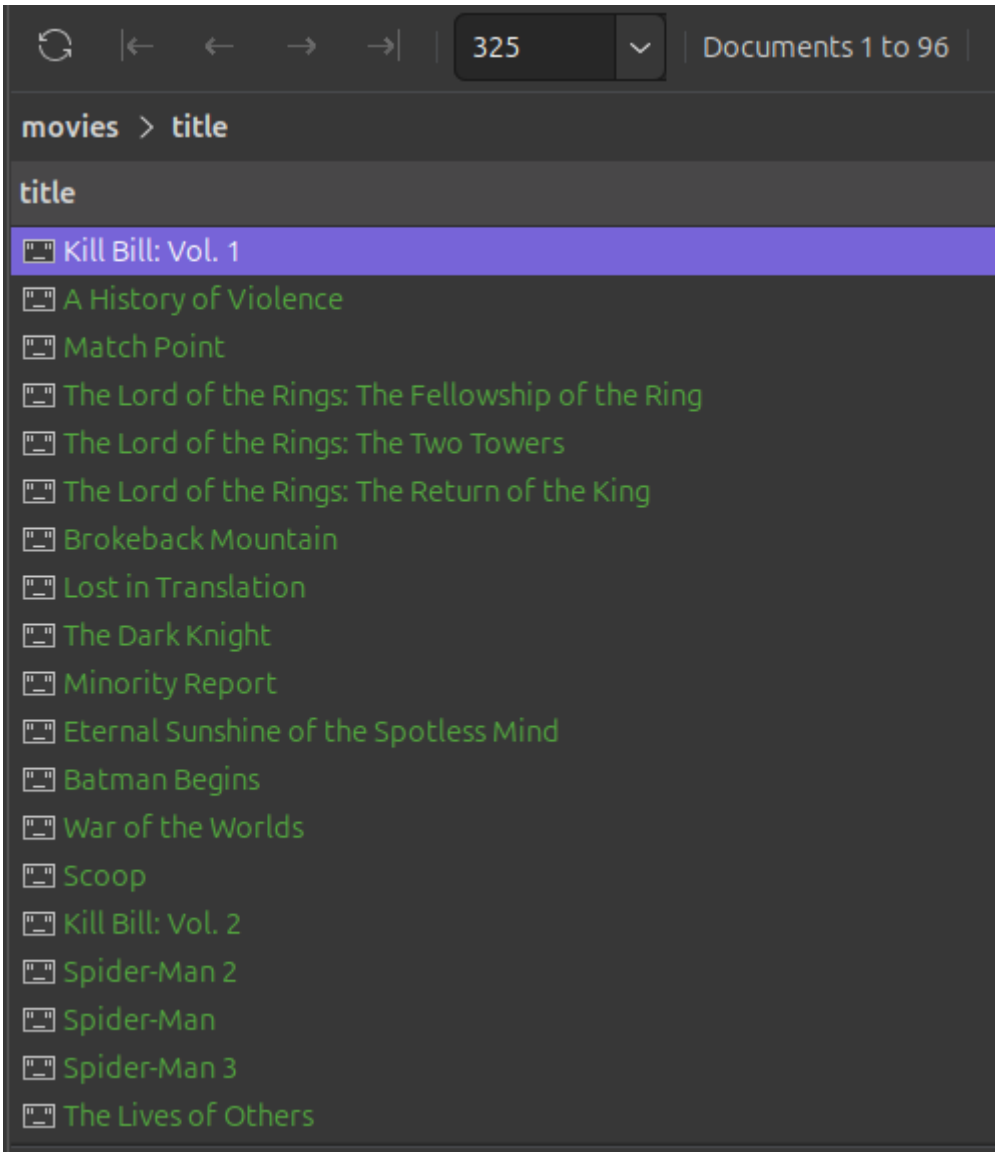
To do this exercise, the **Artists** and **Movies** collections will be used.

a) All titles of movies




```
db.movies.find({}, {_id: 0, title:1})
```

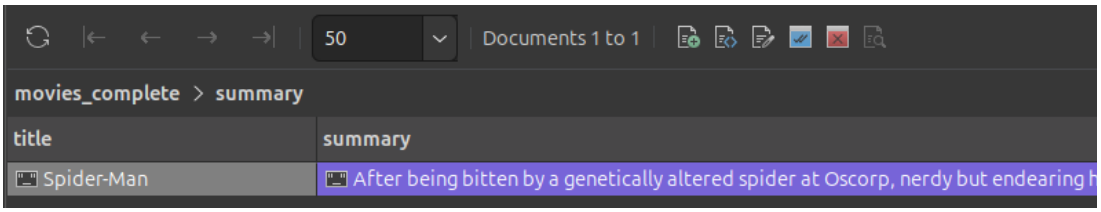
b) All the titles of movies released after 2000 and before 2019




```
db.movies.find({year: {$gt:2000, $lt: 2019}}, {_id: 0, title:1})
```

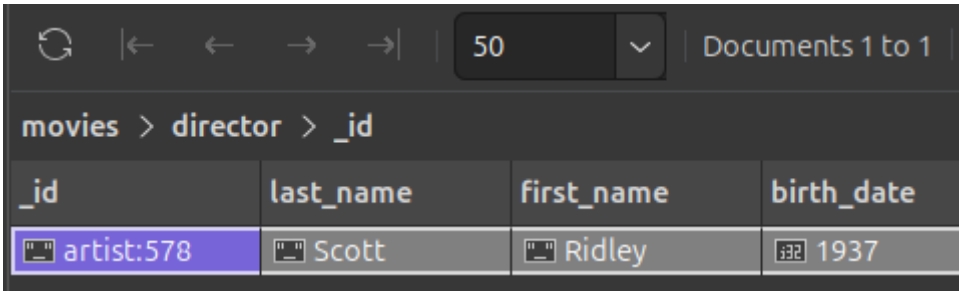
96 movies meet this condition.

c) The summary of Spider-Man movie




```
db.movies.find({title: "Spider-Man"}, {_id: 0, title:1, summary:1})
```

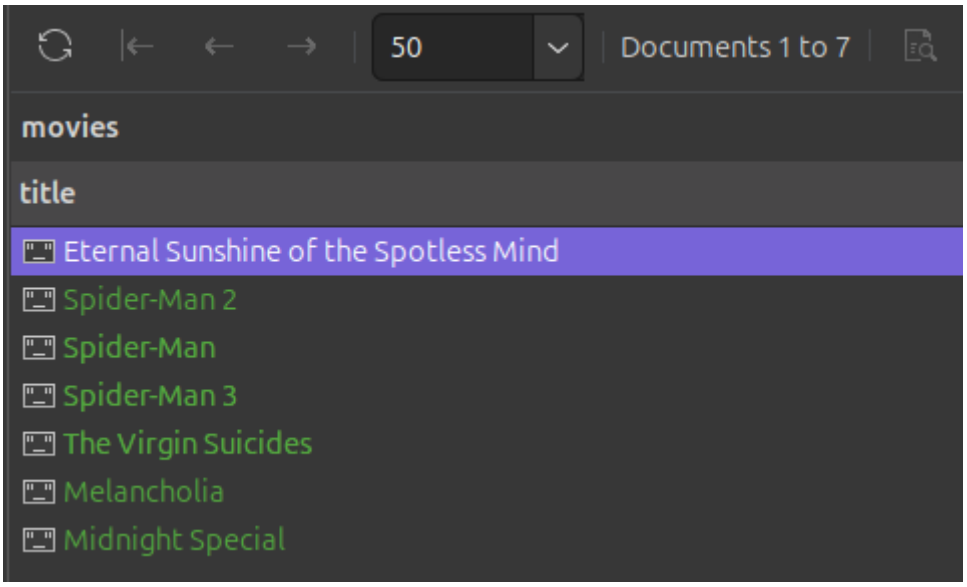
d) Who is the director of Gladiator




```
db.movies.find({title:"Gladiator"}, {_id:0, title:1, director:1})
```

Doing it like it was considered in Exercise 7.

e) Titles of films with Kirsten Dunst




```
db.movies.aggregate([
 {$lookup:{from:"artists", localField:"actors._id", foreignField:"_id",
as:"acts"}},
 {$match:{acts:{$elemMatch:{first_name:"Kirsten", last_name:"Dunst"}}}},
 {$project:{_id:0, title:1}}
])
```

Unlike the directors, the actors do not have their first and last names in the **Movies** collection, therefore it is necessary to use their id from **Movies** to join this collection to the **Artists** one on the id.

f) How many movies have a summary?

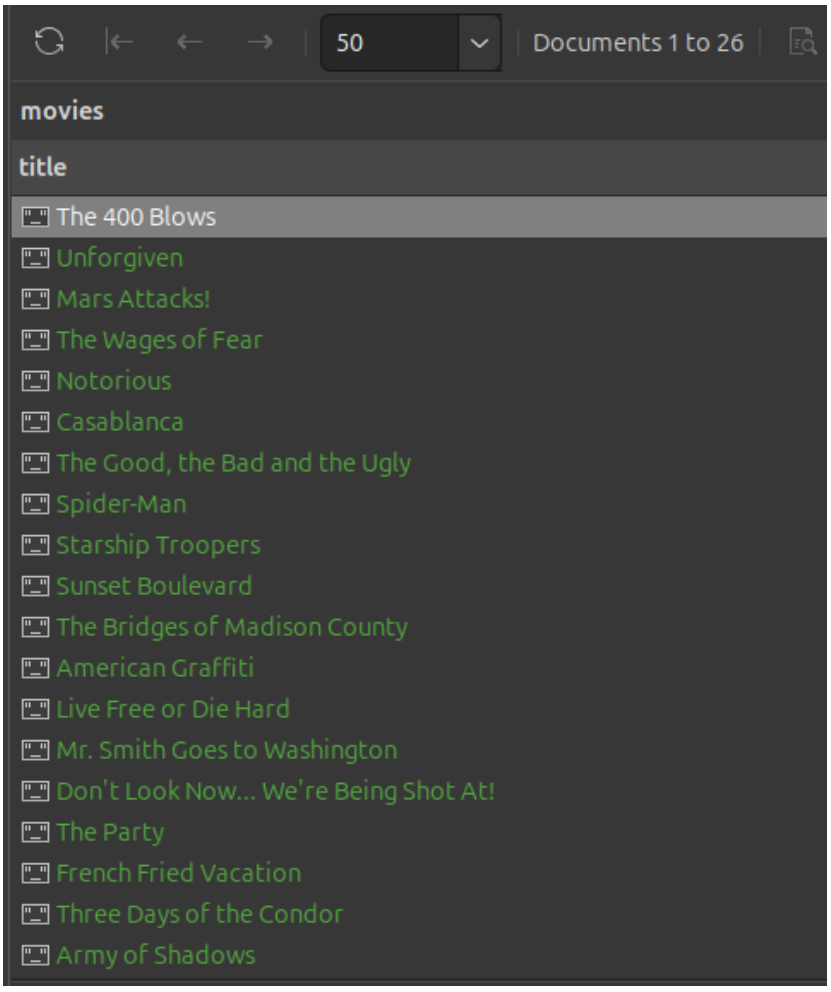
```
db.movies.find({summary:{$ne: null}}).count()
```

$ne (not equal) is used here, another way to do it with $eq (equal) can be done :

```
db.movies.find({}).count() - db.movies.find({summary:{$eq:
null}}).count()
```

Basically doing the difference between the total count of movies and the number of movies with no summary. Both queries give **325 movies**.

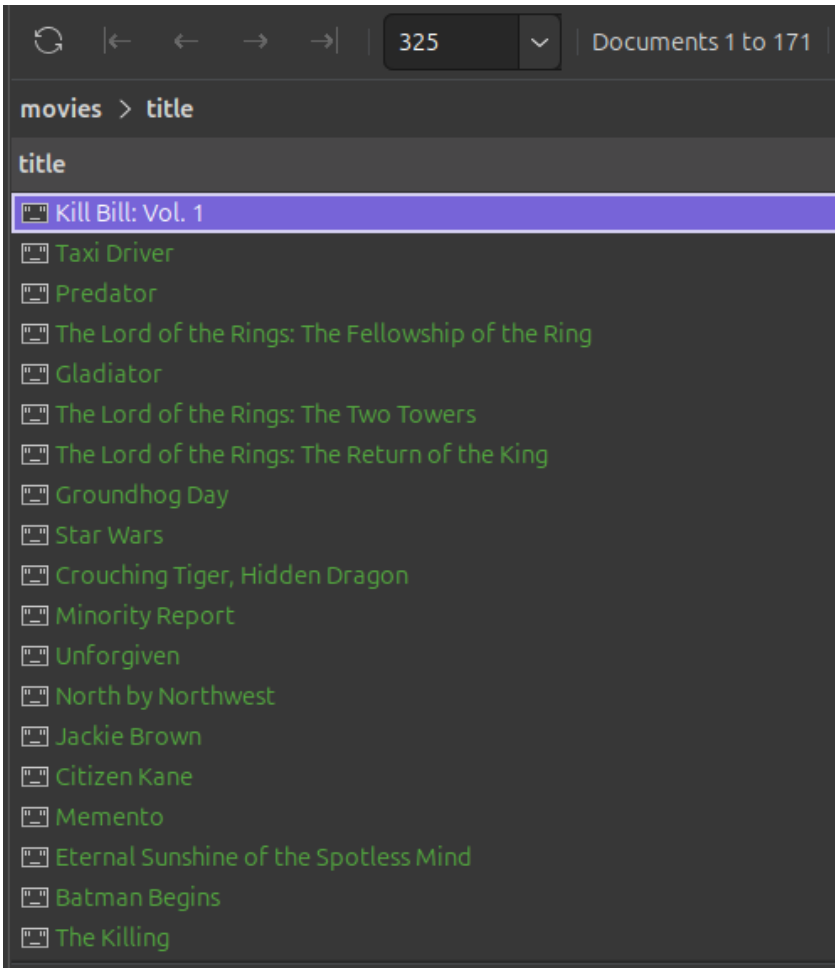
g) What are movies with actors' names that start with "Cl" ?




```
db.movies.aggregate([
 {$lookup:{from:"artists", localField:"actors._id", foreignField:"_id",
as:"acts"}},
 {$match:{acts:{$elemMatch:{$or:[{first_name:/^Cl/},
{last_name:/^Cl/}]}}}},
 {$project:{_id:0, title:1}}
])
```

Another join between two collections was necessary here to get the first and last names. An OR statement was used for the first and last names starting with "Cl", as the question did not specify which one to search. 26 movies meet this condition.

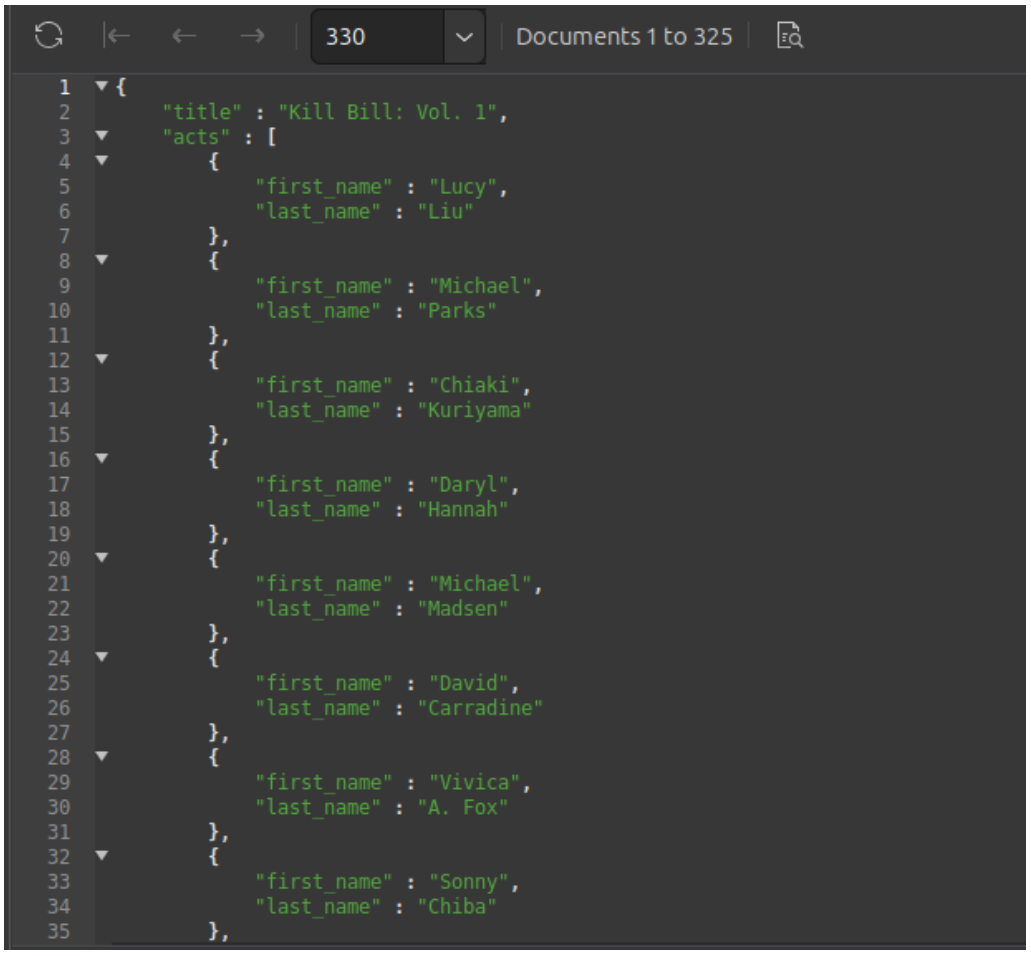
h) Movies that are neither dramas nor comedies.




```
db.movies.find({$nor:[{genre:"Comedy"}, {genre: "Drama"}]}, {_id:0,
title:1})
```

171 movies are neither dramas nor comedies.

i) Titles of the films and the names of the actors.




```
db.movies.aggregate([
 {$lookup:{from:"artists", localField:"actors._id", foreignField:"_id",
as:"acts"}},
 {$project:{_id:0, title:1, "acts.first_name":1, "acts.last_name": 1}}
])
```

The table format was not showing the data as intended, so the visualization got replaced by the .json format :

```
[
 {title : Starwars, actors : [{"last_name":...}]},
 {title : Spider-Man, actors : [{"last_name": Maguire, ...}]}
]
```

The projection could have been different if `act:1` was used instead of `act.first_name:1` and `act.last_name:1` to get all information about the actors.

j) In which films is Clint Eastwood an actor but not a director

```
db.movies.aggregate([
 {$lookup: {from: "artists", localField: "actors._id", foreignField:
"_id", as: "acts"}},
 {$match: {$and: [
   {acts: {$elemMatch: {first_name: "Clint", last_name: "Eastwood"}}},
   {$or: [{"director.last_name": {$ne: "Eastwood"}},
{"director.first_name": {$ne: "Clint"}}]}
 ]}},
 {$project: {_id: 0, title: 1}}
])
```

The Good, the Bad and the Ugly - Sergio Leone

As *directors belongs* to the **Movies** collection, no join was needed to get the director's first and last names, however it was necessary for *actors*. Therefore, every movie with "Clint Eastwood" is first retrieved, then the movies with directors whose first and last names are not "Clint" or "Eastwood" are found.